SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	M.V. Raj Varun	Register Number	192411007
Department / Section	CSE	Academic Year / Semester	MS2 Sep-Dec 2026
Faculty Name	Nattar Kannan	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	06-11-2026	Evaluation Date	06-11-2026

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks

8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required

Failure case	P	HTTP 404 test case documented; actual execution record required
Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required

Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and online-exam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university
- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

Answer:

1.1 Introduction

The university runs five applications (SIS, LMS, Library, Exam Portal, Email) on 12 dedicated, ageing servers in one data centre. Average CPU use is about 15%, yet peaks exceed 95% during admissions and exams. This task defines cloud computing, selects a deployment model and maps each cloud characteristic to the university's real problems.

1.2 Definition of Cloud Computing

Using the NIST definition, cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (networks, servers, storage, applications, services) that can be rapidly provisioned and released with minimal management effort or provider interaction. In plain terms, the university would rent computing as a metered utility instead of owning fixed servers.

1.3 The Five Essential Characteristics

Characteristic	Meaning	University example
On-demand self-service	Provision resources without human interaction with the provider	IT staff launch a server in minutes instead of waiting 6–8 weeks
Broad network access	Services reachable over the network from many device types	Students use browsers and a mobile app on all 4 campuses

Characteristic	Meaning	University example
Resource pooling	Provider resources are shared among many tenants via multi-tenancy	SIS, LMS and Exam Portal draw from one shared pool
Rapid elasticity	Capacity scales out and in automatically	Extra servers during admissions, released afterwards
Measured service	Usage is metered and billed	Pay only for exam-week capacity actually consumed

1.4 Deployment Models

- **Public cloud:** owned by a provider, shared by many organisations (e.g., AWS, Azure). Low cost and high elasticity, but less control.
- **Private cloud:** operated for one organisation, on-premises or hosted. Maximum control, but the organisation carries capital and skills cost.
- **Hybrid cloud:** a combination of private and public, linked by secure connectivity, allowing workloads to be placed by sensitivity and demand.
- **Community cloud:** shared by organisations with common concerns (for example, a consortium of universities sharing compliance needs).

1.5 Deployment Model Suited to the University

A **hybrid cloud** is recommended. Sensitive student records (SIS database) stay in a controlled private or dedicated environment, while bursty, internet-facing components (web tier, exam portal front end, email, static content) run on the public cloud. This respects data-privacy duties, handles peaks through public elasticity and avoids buying hardware for the rare peak. A community cloud with partner colleges is a later option.

1.6 Three Problems of the Current Setup and the Cloud Characteristic that Addresses Each

1. **Peak overload with idle capacity otherwise** (15% average, 95%+ peak): solved by *rapid elasticity*, which adds capacity during peaks and removes it afterwards, and by *resource pooling*, which lets idle capacity be shared.
2. **Slow procurement (6–8 weeks) and high CapEx:** solved by *on-demand self-service* and *measured service*, which convert CapEx to pay-per-use OpEx and provision within minutes.
3. **Monolithic apps with no public interface, and no DR site:** solved by *broad network access* (web and mobile access via APIs) and by the provider's multi-zone/region capacity, which supports a low-cost disaster-recovery site.

1.7 Benefits of Cloud Adoption

- Lower upfront cost and better utilisation (pay for what is used).
- Elastic handling of admissions and result-day spikes.
- Faster delivery of new services, including a mobile app.
- Built-in redundancy, backups and DR options.
- Reduced hardware maintenance workload for the IT team.

1.8 Limitations of Cloud Adoption

- **Data privacy and compliance:** student data stored off-premises needs encryption and contractual safeguards.
- **Vendor lock-in:** proprietary services make later migration costly.
- **Dependence on connectivity:** campus internet outages interrupt access.
- **Skills gap:** staff need cloud, security and cost-management skills.
- **Cost surprises:** poorly governed usage can exceed budget.

1.9 Proposed Cloud Computing Model (Summary Mapping)

University issue	Cloud answer
Idle servers, peak failures	Auto-scaling in a hybrid cloud
Procurement delays	Self-service provisioning
No APIs for mobile/partners	Web-service layer behind an API gateway
No DR	Multi-zone deployment plus backup region
Sensitive data	Private/dedicated database tier with encryption

1.10 Conclusion

A hybrid cloud that uses all five essential characteristics directly resolves the university's three core problems: poor utilisation, slow and costly procurement and lack of openness and resilience. The main limitations (privacy, lock-in, skills) are manageable through encryption, open standards and training, and are examined further in Task 10.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

Answer:

2.1 Introduction

Cloud computing is the result of decades of incremental change in how computing is built and delivered. Each stage removed a limitation of the one before it. This task traces that path, links it to hardware progress and places the university's current system on the timeline.

2.2 Timeline of Computing Evolution

Period	Stage	Key idea
1950s–60s	Mainframes	Centralised, time-shared computing through dumb terminals
1970s	Minicomputers	Smaller, cheaper departmental machines
1980s	Personal computers	Computing on every desk, local software
1990s	Client–server	Shared servers with networked clients
1990s–2000s	Clusters	Many commodity machines acting as one
2000s	Grid computing	Resources pooled across organisations for large jobs
2000s	Utility computing	Metered, pay-per-use computing
Mid-2000s onward	Cloud computing	Elastic, self-service, virtualised utility on the internet

2.3 From Mainframes to Personal Computers

Mainframes were powerful but costly and shared through queues. Minicomputers brought affordable departmental computing. PCs gave individuals independent computing, but data and software became scattered and hard to share or maintain, which created the need for networked server-based systems.

2.4 Client–Server and Clusters

Client–server centralised data and business logic on servers while clients provided the interface, improving sharing and control. However, one application per server led to low utilisation and scalability limits. Clusters of commodity servers provided scalability and fault tolerance at lower cost than a single big machine.

2.5 Grid and Utility Computing

Grid computing pooled geographically distributed resources for large scientific problems but was complex to use and administer. Utility computing introduced the commercial idea of metered consumption like electricity, which set the billing and consumption model for cloud.

2.6 The Cloud Stage

Virtualisation, web services and fast broadband allowed providers to offer elastic, self-service resources at huge scale. Cloud combined the scalability of clusters, resource sharing from grids and the pay-per-use model of utility computing, delivered through a simple web interface or API.

2.7 Hardware Evolution that Enabled the Cloud

- **Multi-core processors with hardware virtualisation support** (Intel VT-x, AMD-V) enable efficient consolidation of many VMs per host.
- **Commodity x86 servers** replaced expensive proprietary systems, making large fleets affordable.
- **High-capacity, low-cost storage** (HDD arrays, then SSD/NVMe) supports big shared storage services.
- **High-speed networking** (Gigabit to 10/25/100 Gbps Ethernet, fibre broadband) makes remote services responsive.
- **Modern data centres** with efficient power, cooling and automation provide economies of scale.

2.8 How Each Stage Solved the Limitation of the Previous One

Stage	Limitation it solved	New limitation it created
Mainframe	Manual, batch computing	High cost, centralised access
Minicomputer/PC	Cost and individual access	Scattered data, hard to share
Client–server	Data sharing and control	One app per server, low utilisation
Cluster	Scalability, fault tolerance	Fixed in-house capacity
Grid	Large cross-site resource pooling	Complex, no commercial billing
Utility/Cloud	Elasticity, metering, self-service	Privacy, lock-in, connectivity dependence

2.9 Stage Corresponding to the University's Current System

The university corresponds to the **client–server era**: dedicated physical servers per application, monolithic software, fixed capacity, manual procurement and no virtualisation or elasticity. The 15% average utilisation is the characteristic weakness of this stage, which is why virtualisation and cloud are the natural next step.

2.10 Conclusion

Cloud computing is the culmination of cheaper hardware, better networks, virtualisation and the utility-billing idea. The university's client–server setup sits two stages behind, and migrating moves it directly to the elastic, shared model that fits its seasonal demand.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet

- Comparison between desktop software, client–server software and web-delivered software
- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

Answer:

3.1 Introduction

Cloud services are delivered as internet software, so understanding how web software evolved explains why applications can now be reached from any device without local installation. This task compares software delivery models and ties them to the university's application design.

3.2 Static Web to Dynamic Web

Early web pages (Web 1.0) were static HTML documents. Server-side technologies (CGI, PHP, JSP, ASP.NET) then generated pages from databases, allowing logins, results lookup and online forms. This made web-based student systems possible.

3.3 Web 2.0 and Rich Interaction

Web 2.0 added user participation and responsive interfaces through AJAX, which updates part of a page without reloading it. Social, collaborative and real-time features followed, raising user expectations of smooth, app-like experiences.

3.4 Service-Oriented Architecture and Mashups

SOA structures software as loosely coupled services with published interfaces (SOAP/WSDL, later REST). Mashups combine services from different sources, for example showing campus bus data on a map. Reusable services let the university expose functions such as results and fees to other systems.

3.5 Cloud-Hosted Applications

Applications and their data now live on provider infrastructure and are accessed by browsers or mobile apps. Delivery models such as SaaS, along with microservices and serverless functions, extend SOA with elastic scaling and subscription billing.

3.6 Role of Browsers, HTTP, AJAX and APIs

- **Browser:** universal client that runs the interface with no installation.
- **HTTP/HTTPS:** stateless request–response protocol carrying all web traffic, secured by TLS.
- **AJAX:** asynchronous calls for partial page updates, giving responsive interfaces.
- **APIs:** programmatic interfaces (usually REST/JSON) that let mobile apps, partners and other systems consume the same services.

3.7 Comparison of Software Delivery Models

Aspect	Desktop software	Client–server	Web-delivered (cloud)
Installation	On every PC	Client install, central server	None (browser/app)
Updates	Manual per machine	Central server, client upgrades	Instant, central
Access	One device	Internal network	Anywhere with internet
Scalability	None	Limited by server	Elastic
Collaboration	Poor	Moderate	Strong, real-time
Maintenance effort	High	Moderate	Low for users

3.8 Effect on the University's Application Design

The monolithic applications bundle everything into single programs on dedicated servers. Web-delivered design suggests separating them into tiers (web, application, database), exposing functions through APIs and making the web tier stateless so it can be cloned for auto-scaling. This also enables a mobile app and partner access without rewriting the core logic.

3.9 Practical Design Implications

- Use REST/JSON APIs for mobile and partner integration.
- Keep sessions in a shared cache or tokens, not on a single server.
- Move static content (notices, documents) to storage with a CDN.
- Refactor gradually, starting with high-demand modules (results lookup, fee payment).
- Apply HTTPS and token-based authentication everywhere.

3.10 Conclusion

The progression from static pages to API-driven, cloud-hosted software gives the university a path to deliver services to browsers, mobile apps and partner colleges from elastic infrastructure. Its monolithic design is the main obstacle and should be decomposed in phases to benefit from the cloud.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization
- Propose which applications should run as virtual machines and which as containers, with reasons

- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

Answer:

4.1 Introduction

Twelve servers at about 15% average utilisation waste hardware, power and space. Virtualization lets several isolated workloads share one physical host, so the university can consolidate onto a few well-used hosts. This task explains the technology, assigns workloads, and shows the consolidation calculation.

4.2 How a Hypervisor Works; Type-1 and Type-2

A hypervisor (virtual machine monitor) sits between hardware and guest operating systems. It presents each virtual machine (VM) with virtual CPUs, memory, disk and network, schedules vCPUs on physical cores, maps guest memory to physical memory and mediates device access, keeping VMs isolated.

Feature	Type-1 (bare-metal)	Type-2 (hosted)
Runs on	Directly on hardware	As an application on a host OS
Performance	Near-native, low overhead	Lower, extra OS layer
Typical use	Data centres, production	Desktops, labs, testing
Examples	VMware ESXi, Microsoft Hyper-V, Xen, KVM	VirtualBox, VMware Workstation

The university should use a Type-1 hypervisor in production.

4.3 Comparison of Virtualization Techniques

Technique	How it works	Strength	Weakness
Full virtualization	Hypervisor emulates hardware; unmodified guest OS	Runs any OS	Trap/emulation overhead

Technique	How it works	Strength	Weakness
Para-virtualization	Guest OS modified to call hypervisor (hypercalls)	Lower overhead, e.g., I/O	Needs modified guest
Hardware-assisted	CPU features (VT-x, AMD-V, EPT) trap privileged ops in hardware	Near-native, no guest change	Needs modern CPUs
OS-level (containers)	Processes isolated by kernel namespaces and cgroups	Fast start, very dense	Shared kernel, weaker isolation

Modern production combines hardware-assisted full virtualization with para-virtual drivers (virtio, VMware tools).

4.4 Which Applications Run as VMs and Which as Containers

Application	Choice	Reason
SIS (database and core logic)	VM	Sensitive data, strong isolation, stateful
Exam Portal database	VM	Integrity critical, isolation from other workloads
Email server	VM	Legacy, stateful, hard to containerise
LMS web/application tier	Containers	Stateless, scales out fast during exams
Library system web tier	Containers	Light, low-risk, dense packing
New REST/API services	Containers	Quick deployment and horizontal scaling

4.5 Consolidation Assumptions

- Each of the 12 servers has equal capacity of one "server unit" (1 SU) and averages 15% utilisation.
- Target host utilisation is 70%, leaving 30% headroom for spikes and hypervisor overhead.
- The new hosts have the same capacity as the old servers (conservative; modern hosts usually have more).
- Hypervisor overhead is assumed to be 10% on top of workload demand.
- One extra host is kept for failover (N+1).

4.6 Consolidation Calculation

Average aggregate demand:

$$D = 12 \times 0.15 = 1.8 \text{ SU}$$

With 10% virtualization overhead:

$$D' = 1.8 \times 1.10 = 1.98 \text{ SU}$$

Hosts needed at 70% target utilisation:

$$N = \frac{D'}{0.70} = \frac{1.98}{0.70} = 2.83 \approx 3 \text{ hosts}$$

Adding one host for failover (N+1) gives **4 physical hosts**, replacing 12 servers (a 3:1 reduction, about 67% fewer machines).

4.7 Peak-Load Caution

The average calculation does not cover peaks. If all applications hit 95% together, demand would be 12 × 0.95 × 1.10 = 12.5 SU, needing about 18 hosts at 70% – uneconomic to own. In practice, peaks come from different applications at different times (SIS at admissions, Exam Portal and LMS at exams), and stateless tiers can burst to the public cloud. The 4-host base therefore handles normal load, and elasticity covers peaks (Tasks 8 and 11).

4.8 Benefits: Live Migration, Snapshots and Over-commitment

- **Live migration** moves a running VM between hosts with near-zero downtime, enabling maintenance without outages.
- **Snapshots** capture VM state for quick rollback before upgrades and for backup.
- **Over-commitment** (allocating more virtual CPU/RAM than physical) raises utilisation because workloads rarely peak together; a moderate CPU ratio such as 2:1 is safe, but memory should be overcommitted cautiously.
- Faster provisioning, hardware independence and simpler disaster recovery.

4.9 Risks of Virtualization

- **Noisy neighbour:** one busy VM starves others of CPU, I/O or network; mitigated by resource limits, reservations and placement rules.
- **Hypervisor failure:** a single host failure takes down many VMs; mitigated by clustering, HA restart and N+1 capacity.
- **Security isolation:** VM escape, side-channel attacks and shared-kernel weaknesses in containers; mitigated by patching, separating sensitive workloads and network segmentation.
- **VM sprawl:** unmanaged VMs consume capacity and stay unpatched.

4.10 Conclusion

Virtualizing onto Type-1 hypervisors consolidates 12 servers into about 4 hosts (3 for load plus 1 spare). Stateful, sensitive systems go in VMs and stateless web tiers in containers. Because the arithmetic uses averages, the plan relies on elasticity for peaks and must be validated by load testing.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)
- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

Answer:

5.1 Introduction

The applications currently have no public interfaces, which blocks mobile apps and partner integration. Web services expose university functions as machine-readable interfaces over HTTP so that other systems can call them. This task defines the services, compares SOAP and REST, and specifies three APIs.

5.2 Web Services and Their Role in Cloud and SOA

A web service is a software function offered over a network using standard protocols. In SOA, applications are built from loosely coupled services that publish contracts; in the cloud, these services are hosted on elastic infrastructure, scaled independently and accessed by many client types (browser, mobile, partner systems).

5.3 SOAP, WSDL and UDDI

- **SOAP:** an XML-based messaging protocol with a strict envelope, built-in standards for security (WS-Security) and reliability, typically over HTTP.
- **WSDL:** an XML document that describes a service's operations, messages and endpoints, so tools can generate client code.
- **UDDI:** a registry for publishing and discovering services; rarely used today.

5.4 RESTful Services

REST treats everything as a resource identified by a URL and manipulated with HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove). Data is usually JSON (sometimes XML). REST is stateless, cache-friendly, lightweight and easy to consume from mobile and web.

5.5 SOAP versus REST

Aspect	SOAP	REST
Format	XML only	JSON, XML, others
Contract	WSDL, strict	OpenAPI (optional), flexible
Weight	Heavy	Light
Security	WS-Security, message-level	HTTPS, OAuth 2.0, tokens
Transactions	Built-in standards	Handled in application logic
Best for	Enterprise, formal integrations	Web and mobile, public APIs

5.6 Service 1: Result Lookup

- **Endpoint/method:** `GET /api/v1/students/{regNo}/results?semester=5`
- **Sample request:** `GET /api/v1/students/21CS001/results?semester=5` with header `Authorization: Bearer <token>`
- **Sample response (200):**

```
{"regNo":"21CS001","semester":5,"gpa":8.4,"subjects":[{"code":"CSA15","grade":"A"}
]}
```

- **Choice: REST.** Read-only, high-volume, consumed by the mobile app; JSON and caching suit it.

5.7 Service 2: Fee Payment

- **Endpoint/method:** `POST /api/v1/fees/payments`
- **Sample request:**

```
{"regNo":"21CS001","amount":45000,"currency":"INR","term":"2025-ODD"}
```

- **Sample response (201):**

```
{"paymentId":"PAY-88231","status":"SUCCESS","receiptUrl":"/api/v1/fees/receipts/PA
Y-88231"}
```

- **Choice: REST** for the student-facing app (with OAuth 2.0, TLS and idempotency keys to prevent double charges). A **SOAP** interface is justified only for integration with a legacy bank gateway that mandates WS-Security and formal transactions.

5.8 Service 3: Library Search

- **Endpoint/method:** `GET /api/v1/library/books?title=cloud&limit=10`
- **Sample response (200):**

```
{"total":2,"items":[{"isbn":"9780134...","title":"Cloud
Computing","available":3}]}
```

- **Choice: REST.** Simple, public, read-only, cacheable and needing no strict contract.

5.9 Summary of Service Choices

Function	Endpoint	Method	Format	Justification
Result lookup	/students/{regNo}/results	GET	REST/JSON	Read-only, mobile-friendly
Fee payment	/fees/payments	POST	REST/JSON (SOAP for bank link)	Light client, secure tokens
Library search	/library/books	GET	REST/JSON	Public and cacheable

All services should sit behind an API gateway providing authentication, rate limiting and logging.

5.10 Conclusion

Exposing result lookup, fee payment and library search as REST APIs enables the mobile app and partner access with low overhead, while SOAP is retained only where a formal enterprise contract is demanded. These services form the basis of the architecture in Task 8.

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model
- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lock-in
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

Answer:

6.1 Introduction

Choosing a service model decides how much of the technology stack the university manages and how much the provider manages. The right choice balances control, cost, scalability and effort for each of the five applications. This task defines the models, maps the applications and compares them.

6.2 Definitions

- **IaaS (Infrastructure as a Service):** the provider supplies virtualised compute, storage and networking; the customer manages OS, middleware, runtime and application.
- **PaaS (Platform as a Service):** the provider also manages OS, middleware and runtime; the customer deploys application code and data.
- **SaaS (Software as a Service):** the provider manages everything; the customer simply uses the finished application.

6.3 Who Manages Which Layer

Layer	On-premise	IaaS	PaaS	SaaS
Application	University	University	University	Provider
Data	University	University	University	University (configuration/content)
Runtime / middleware	University	University	Provider	Provider

Layer	On-premise	IaaS	PaaS	SaaS
Operating system	University	University	Provider	Provider
Virtualization	University	Provider	Provider	Provider
Servers, storage, network	University	Provider	Provider	Provider

6.4 Mapping the Five Applications

Application	Model	Main reason
SIS	IaaS	Needs OS-level control, custom integrations and data residency
LMS	PaaS	Custom web app needing auto-scaling without OS administration
Library system	SaaS	Commodity function; hosted library software suffices
Exam Portal	PaaS	Extreme, short-lived peaks; elastic runtime with managed scaling
Email	SaaS	Standard utility; mailbox hosting is best outsourced

6.5 Reasoning: SIS and LMS

SIS → IaaS. It is the system of record, holds sensitive student data, is heavily customised and tied to institutional processes. IaaS gives control over OS, patching, encryption, network placement and database tuning, and lets it be placed in a private or dedicated segment. The price is that the university still manages OS and middleware.

LMS → PaaS. The LMS is a web application with large seasonal peaks. PaaS auto-scales instances and removes OS and runtime maintenance, so the small IT team focuses on course content and code, not servers.

6.6 Reasoning: Library, Exam Portal and Email

Library → SaaS. Cataloguing and circulation are standard needs; a hosted product removes all infrastructure work with minimal customisation need.

Exam Portal → PaaS. Load is near zero for months, then thousands of simultaneous users. PaaS with auto-scaling and managed load balancing fits this pattern and avoids pre-buying capacity; a serverless option for answer submission is also suitable.

Email → SaaS. Running mail servers brings spam, security and availability burdens with no competitive value. A SaaS mail service provides high availability, spam filtering and mobile access.

6.7 Comparison of the Models

Criterion	IaaS	PaaS	SaaS
Control	Highest	Medium	Lowest
Cost model	Pay for VMs, higher admin cost	Pay per usage/instance, lower admin cost	Per-user subscription
Scalability	Manual or scripted	Automatic, built in	Provider-managed

Criterion	IaaS	PaaS	SaaS
Maintenance effort	High	Low to medium	Minimal
Vendor lock-in	Low to medium	Medium to high	High
Customisation	Full	Within the platform	Limited

6.8 Trade-offs of the Chosen Mix

The mix gives control where data is most sensitive (SIS), elasticity where demand is spiky (LMS, Exam Portal) and minimal effort for commodity services (Library, Email). The downside is managing three models, three vendors' interfaces and integration between them; a common identity system and API gateway reduce this.

6.9 Real Provider Examples

- **IaaS:** Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine.
- **PaaS:** Azure App Service, Google App Engine, AWS Elastic Beanstalk.
- **SaaS:** Microsoft 365, Google Workspace, Salesforce.

6.10 Conclusion

No single model suits every application. Mapping SIS to IaaS, LMS and Exam Portal to PaaS, and Library and Email to SaaS balances control, scalability and effort, and reduces the administrative workload of a small IT team while protecting sensitive data.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

Answer:

7.1 Introduction

XaaS (Anything as a Service) generalises the three core models: almost any IT capability can be consumed as a metered, remotely managed service. This task surveys XaaS variants, selects two for the university and discusses the risks of depending on many providers.

7.2 What XaaS Means

XaaS is a collective term for delivering functions, tools or entire processes as subscription or pay-per-use services over the internet. It lets an organisation obtain specialised capabilities without building and operating them, and it extends beyond IaaS, PaaS and SaaS.

7.3 Variant 1: Desktop as a Service (DaaS)

Virtual desktops hosted in the cloud and streamed to any device. For the university, it could give students remote access to lab software (CAD, MATLAB) from personal laptops, reducing computer-lab hardware.

7.4 Variant 2: Backend / Mobile Backend as a Service (BaaS/MBaaS)

Prebuilt backend features (authentication, push notifications, storage, APIs) for apps. It would speed up the planned mobile app by avoiding custom backend development.

7.5 Variant 3: Function as a Service (FaaS)

Event-driven code run without managing servers, billed per execution (AWS Lambda, Azure Functions). Suitable for result-publication notifications and exam-answer submission spikes.

7.6 Variant 4: Database as a Service (DBaaS)

A managed database with automated backups, patching, replication and scaling (Amazon RDS, Azure SQL Database). It removes the heavy administrative load of the SIS and exam databases.

7.7 Variants 5 and 6: Security-as-a-Service and Disaster-Recovery-as-a-Service

- **SECaaS:** outsourced identity management, firewalls, malware scanning and monitoring.
- **DRaaS:** replication of servers and data to a provider site, with failover on demand. The university currently has no DR site, so DRaaS fills a critical gap without building a second data centre.

7.8 Selected Offerings and Justification

1. DBaaS. The SIS and Exam Portal databases are the most critical assets. Managed backups, point-in-time recovery, patching and replication improve availability and free staff from database administration, while encryption at rest and in transit protect records.

2. DRaaS. The university has no recovery site and faces risks of outages during admissions and exams. DRaaS offers pay-as-you-go replication, with costs mainly when a failover happens, which is far cheaper than a second data centre.

7.9 Risks of Over-Reliance on Multiple "as-a-Service" Providers

- **Vendor lock-in** across proprietary APIs and data formats.

- **Integration complexity** and inconsistent security between providers.
- **Cascading outages** when one provider or the internet link fails.
- **Cost sprawl** from overlapping subscriptions and unmonitored usage.
- **Compliance and data-privacy** exposure when data is spread across jurisdictions.
- **Loss of skills** as staff stop practising core operations.

Mitigations: open standards and exportable data, a central identity provider, exit plans and contracts with SLAs, a cost-governance process and regular DR tests.

7.10 Conclusion

XaaS lets the university buy specialised capability instead of building it. DBaaS and DRaaS give the highest value for the risk and effort, provided lock-in and integration risks are governed through standards, contracts and monitoring.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multi-zone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

Answer:

8.1 Introduction

This task combines virtualization (Task 4), web services (Task 5) and the service-model mix (Tasks 6 and 7) into one hybrid architecture. It shows the components, how users reach them, how the system scales for admissions and exam peaks, and how security and availability are protected.

8.2 Architecture Diagram

Figure 8.1 shows how requests travel from users through the edge and API gateway to the elastic web tier and the protected data tier, with a disaster-recovery region behind.

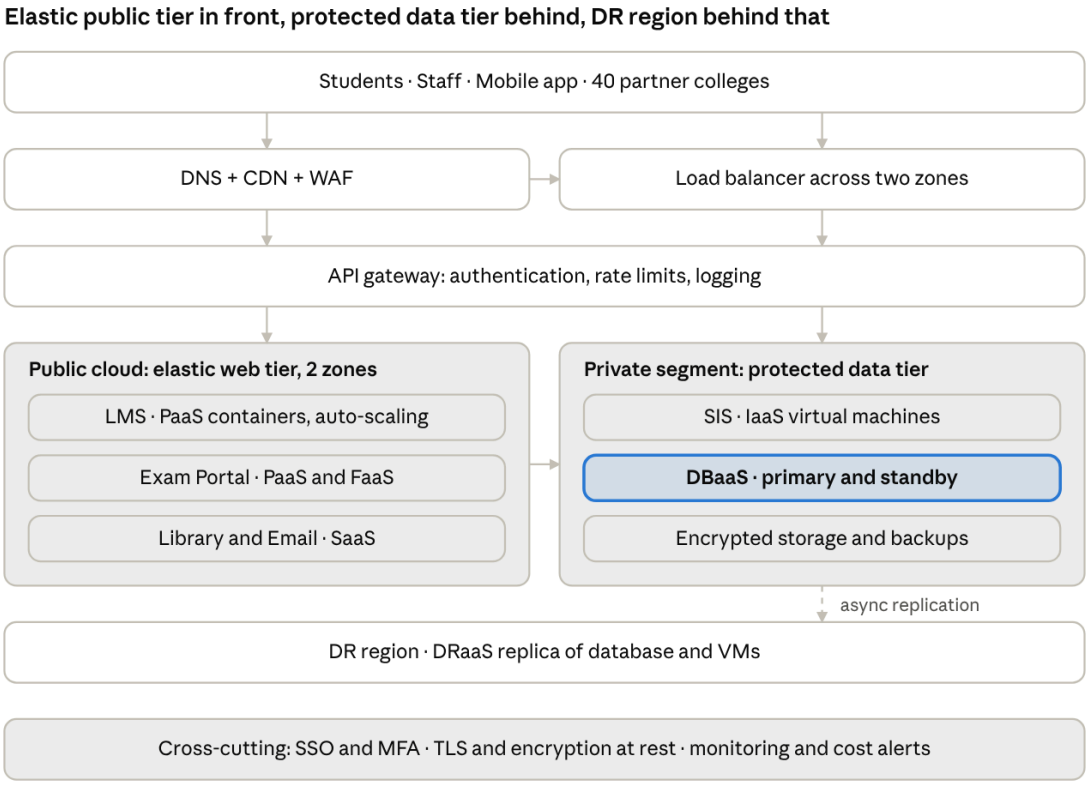


Figure 8.1 · proposed hybrid cloud architecture

Figure 8.1: Proposed hybrid cloud architecture. Solid arrows show request flow; the dashed arrow shows asynchronous replication to the DR region.

8.3 Deployment Model per Component

Component	Deployment model	Reason
CDN, WAF, load balancer, web tier	Public cloud	Elastic, internet-facing, low sensitivity
SIS virtual machines and database	Private/dedicated segment of a hybrid cloud	Sensitive student records, strict isolation
Backups and DR replica	Public cloud, different region	Low cost, geographic separation

8.4 Service Model per Component

Component	Service model	Examples
LMS web/app tier	PaaS (containers)	Azure App Service, Elastic Beanstalk
Exam Portal	PaaS plus FaaS	Auto-scaled app, serverless answer submission
SIS application	IaaS	Virtual machines on a Type-1 hypervisor
Databases	DBaaS (XaaS)	Managed relational service with standby

Component	Service model	Examples
Library and Email	SaaS	Hosted library product, Microsoft 365 or Google Workspace
DR	DRaaS (XaaS)	Replicated VMs and data in a second region

8.5 Request Flow

1. A student opens the browser or mobile app; DNS resolves to the CDN.
2. The CDN serves static files; the WAF filters malicious requests.
3. The load balancer spreads traffic across healthy instances in two zones.
4. The API gateway authenticates the token, applies rate limits and routes the call.
5. The LMS, Exam Portal or SIS service processes it.
6. Data is read from or written to the managed database or encrypted storage.
7. The response returns by the same path; logs go to monitoring.

8.6 Elasticity and Auto-Scaling

The web tier is stateless, with sessions kept in a shared cache or tokens, so any instance can serve any request. An auto-scaling group adds instances when average CPU exceeds a threshold (for example 60%) and removes them when it falls. For known events such as admissions and exam days, **scheduled scaling** raises minimum capacity beforehand to avoid cold-start delays. A maximum limit prevents runaway cost. Static content offloaded to the CDN reduces origin load.

8.7 Security Measures

- **Access control:** role-based access and least privilege; SSO with multi-factor authentication for staff.
- **Encryption:** TLS in transit; encryption at rest for databases, backups and storage with managed keys.
- **Network:** private subnets for the data tier, security groups/firewalls and WAF at the edge.
- **Application:** token-based API authentication (OAuth 2.0) and input validation.
- **Governance:** audit logging, patching, vulnerability scans.

8.8 Availability and Backup

- Two availability zones for web tier and database standby (automatic failover).
- Automated daily backups plus point-in-time recovery, stored in a separate region.
- DR region with replicated data; target examples: RPO of 15 minutes, RTO of 1 hour for SIS (to be agreed with management).
- Health checks and automatic replacement of failed instances.
- Regular restore and failover drills.

8.9 Justification of Key Design Decisions

Decision	Justification
Hybrid cloud	Protects sensitive data and still gets public-cloud elasticity
Stateless web tier + auto-scaling	Handles peaks without owning idle capacity

Decision	Justification
API gateway	One controlled entry for mobile, web and partners
Managed database (DBaaS)	Less administration, built-in backup and failover
Multi-zone plus DR region	Removes single points of failure; supports a high availability target
Mixed IaaS/PaaS/SaaS	Control where needed, low effort elsewhere

8.10 Conclusion

The architecture places each component where its sensitivity and load pattern fit best, uses auto-scaling for peaks, and protects data with layered security, multi-zone deployment and a DR region. It is also the starting point for the Day-4 redesign in Task 11.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

Answer:

9.1 Introduction

This proof-of-concept (PoC) implements the Result Lookup service from Task 5 as a containerised REST API, tests it with a client and describes deployment to a PaaS. All commands are reproducible. **Important:** the outputs and screenshots below are the expected results; run each step yourself and replace the placeholders with your own captured screenshots, as the assignment requires genuine evidence.

9.2 Objective and Scope

Host one university function (result lookup) as a REST API in a Docker container, deploy it to a free-tier PaaS, and verify a success case (HTTP 200) and a failure case (HTTP 404).

9.3 Environment

Item	Value
Operating system	Windows 11 / Ubuntu 22.04 (state yours)
Docker	Docker Desktop or Docker Engine 24+
Language	Python 3.12 with Flask
Test client	curl (and optionally Postman)
PaaS	Render, Azure App Service or Google Cloud Run free tier

Check the tools with `docker --version` and `python --version`.

9.4 Source Code (app.py)

```
from flask import Flask, jsonify

app = Flask(__name__)

RESULTS = {
    "21CS001": {"semester": 5, "gpa": 8.4, "subjects": [{"code": "CSA15", "grade":
"A"}]},
    "21CS002": {"semester": 5, "gpa": 7.9, "subjects": [{"code": "CSA15", "grade":
"B+"}]},
}

@app.route("/health")
def health():
    return jsonify(status="ok")

@app.route("/api/v1/students/<reg_no>/results")
def get_results(reg_no):
    data = RESULTS.get(reg_no)
    if data is None:
        return jsonify(error="Student not found"), 404
    return jsonify(regNo=reg_no, **data), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

A `requirements.txt` file contains `flask==3.0.3` and `gunicorn==22.0.0`.

9.5 Dockerfile, Build and Run

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

```
docker build -t uni-results-api .
docker run -d -p 5000:5000 --name results-api uni-results-api
docker ps
```
Expected: `docker ps` lists the container `results-api` with status "Up" and port mapping `0.0.0.0:5000->5000/tcp`. **Screenshot 1:** build output. **Screenshot 2:** `docker ps`.

9.6 Testing the Web Service

Success case:

```
curl -i http://localhost:5000/api/v1/students/21CS001/results
```
Expected: `HTTP/1.1 200 OK` and body `{"gpa":8.4,"regNo":"21CS001","semester":5,"subjects":[{"code":"CSA15","grade":"A"}]}`.

Failure case:

```
curl -i http://localhost:5000/api/v1/students/99XX999/results
```
Expected: `HTTP/1.1 404 NOT FOUND` and body `{"error":"Student not found"}`.

Screenshot 3: both curl outputs (or the Postman request and response panes).

9.7 PaaS / Serverless Component

Deploy the same repository to a free-tier PaaS (for example Render):

1. Push `app.py`, `requirements.txt` and `Dockerfile` to a GitHub repository.
2. In the PaaS console, create a new Web Service, connect the repository and choose the Docker runtime.
3. Set the health-check path to `/health` and deploy.
4. When the status is Live, call `https://<your-app>.onrender.com/api/v1/students/21CS001/results` with curl or a browser.

Alternative: create a serverless function (AWS Lambda behind API Gateway) or a static site on object storage. **Screenshot 4:** deployment dashboard showing "Live". **Screenshot 5:** the public URL response.

9.8 Evidence Checklist

No.	Evidence	Status
1	Docker build output	[Insert your screenshot]
2	Running container (docker ps)	[Insert your screenshot]
3	curl 200 and 404 outputs	[Insert your screenshot]
4	PaaS deployment page	[Insert your screenshot]
5	Public URL response	[Insert your screenshot]

9.9 Issues Faced and Fixes

- **Port already in use:** change the mapping, e.g. `-p 5001:5000`.

- **Container exits immediately:** read `docker logs results-api`; usually a missing dependency or a typo in `app:app`.
- **App unreachable from the host:** the Flask host must be `0.0.0.0`, not `127.0.0.1`.
- **PaaS cold start:** free tiers sleep when idle, so the first request is slow.

Record any issues you actually encounter here.

9.10 Conclusion

The PoC shows that a university function can be packaged as a container, exposed through a REST endpoint, tested for both success and error paths and deployed to a PaaS with little operational effort. The same pattern scales to the fee-payment and library APIs. It uses in-memory sample data, so a production system would use DBaaS and authentication.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort
- Estimate a simple 3-year cost comparison using stated assumptions
- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

Answer:

10.1 Introduction

This task compares three ways of running the university's applications, estimates a three-year cost, assesses risks and recommends a final solution with a migration roadmap. The recommendation follows from the analysis in Tasks 1 to 9.

10.2 Options Evaluated

- **Option A – On-premise refresh:** replace the ageing servers with new ones in the same data centre.
- **Option B – IaaS-based:** lift-and-shift all applications to cloud virtual machines.

- **Option C – PaaS/SaaS-led hybrid:** SIS on IaaS with DBaaS, LMS and Exam Portal on PaaS, Library and Email on SaaS, plus DRaaS.

10.3 Qualitative Comparison

Criterion	A: On-premise	B: IaaS	C: PaaS/SaaS-led hybrid
Cost model	High CapEx	OpEx, steady	OpEx, usage-based
Scalability	Fixed; peaks still fail	Good but manual or scripted	Automatic, best for peaks
Availability	Single site, no DR	Multi-zone possible	Multi-zone plus DRaaS
Security control	Full, but self-managed	Shared responsibility	Shared; provider secures platform
Operational effort	Highest	High (OS, patching)	Lowest
Speed of delivery	6–8 week procurement	Minutes	Minutes
Lock-in risk	None	Low	Medium to high

10.4 Cost Assumptions

All figures are illustrative planning estimates in ₹ lakh and should be replaced with real vendor quotes.

- **A:** CapEx ₹46 lakh (servers 36, storage and network 10); annual power, cooling and space 8; AMC 5; two administrators at 6 each = 12.
- **B:** migration ₹8 lakh; cloud compute, storage and network ₹2.0 lakh/month = 24/yr; backup/DR replication 3/yr; 1.5 administrators = 9/yr.
- **C:** migration and refactoring ₹15 lakh plus training 2; PaaS, DBaaS and IaaS 14.4/yr; SaaS subscriptions 8/yr; DRaaS and backup 3/yr; one administrator 6/yr.
- Three-year horizon; no discounting; inflation ignored.

10.5 Three-Year Cost Comparison

Item (₹ lakh)	A: On-premise	B: IaaS	C: Hybrid
One-time (CapEx / migration / training)	46.0	8.0	17.0
Annual running cost	25.0	36.0	31.4
Running cost for 3 years	75.0	108.0	94.2
3-year total	**121.0**	**116.0**	**111.2**

Working for C: 14.4 + 8 + 3 + 6 = 31.4 per year; 31.4 × 3 = 94.2; 94.2 + 17 = 111.2.

10.6 Interpretation of the Cost Result

Option C is about 8% cheaper than A (111.2 vs 121.0) and about 4% cheaper than B. The gap is modest, so cost alone does not decide. The key difference is that A buys fixed capacity that still fails

at peak and has no DR, while C includes automatic scaling and DR in its price. Covering peak demand and DR on-premise would raise A's cost substantially.

10.7 Risks and Mitigation

Risk	Effect	Mitigation
Vendor lock-in	Costly to switch	Containers, open APIs, standard databases, exportable data, exit clause
Data privacy	Student records exposed or non-compliant	Encryption, India-region hosting, access control, contracts, audit logs
Downtime	Failed admissions or exams	Multi-zone deployment, DRaaS, SLAs, scheduled scaling, failover drills
Skills gap	Misconfiguration, cost overruns	Training and certification, managed services, a small cloud team, a partner for initial setup
Cost overrun	Budget exceeded	Budgets and alerts, auto-scaling caps, reserved capacity

10.8 Final Recommendation

Adopt **Option C, a PaaS/SaaS-led hybrid cloud**. It is the lowest-cost option on the stated assumptions, is the only option that scales automatically for admissions and exam peaks, provides DR, reduces administration and keeps the sensitive SIS in a controlled IaaS/DBaaS segment. The choice rests on the mapping in Task 6, the architecture in Task 8 and the PoC in Task 9, not on provider popularity. Any provider chosen should be selected through a comparison of price, India-region availability, SLA and exit terms.

10.9 Migration Roadmap

Phase	Timing	Move	Why in this order
0	Month 0–1	Cloud account, identity and network, cost controls, staff training	Foundation for everything else
1	Month 1–2	Email and Library to SaaS	Low risk, quick win, immediate relief
2	Month 2–4	LMS to PaaS with CDN	Large seasonal peaks; stateless and easy to scale
3	Month 4–6	Exam Portal to PaaS and FaaS	Must be live before the next exam season; reuses LMS lessons
4	Month 6–9	SIS to IaaS with DBaaS; enable DRaaS	Most sensitive and complex, so last, after experience is gained
5	Month 9–10	Decommission old servers; review cost	Realise savings

10.10 Conclusion

The PaaS/SaaS-led hybrid gives the best balance of cost, scalability, availability and manageability for the university. By migrating low-risk applications first and the SIS last, and by governing lock-in, privacy, downtime and skills risks, the university can move to the cloud safely and within budget.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

Answer:

11.1 Introduction

The university will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10× the current peak, availability must be 99.9% and the budget is limited. This task analyses the impact (2 marks), evaluates the Task 8 design (2 marks), decides to retain, modify or redesign (2 marks) and defends the decision with numbers (2 marks).

11.2 Impact Analysis: What Has Changed

Aspect	Original scenario	New scenario
Users	25,000 students, 2,000 staff, 4 campuses	Plus mobile users and 40 affiliated colleges

Aspect	Original scenario	New scenario
Peak load	1× (admissions and exams)	10× the current peak
Availability	Not formally defined, no DR	99.9% (8.76 hours of downtime per year)
Tenancy	One organisation	40 external organisations sharing services
Budget	Capital-heavy	Limited operating budget

11.3 Affected Assumptions and Why APIs Become Critical

The load assumption (seasonal peaks of one institution), the user assumption (trusted internal users) and the availability assumption (best effort) no longer hold, and a fixed-size database or an instance cap sized for today's peak would be overwhelmed. APIs become critical because the mobile app and college systems do not use the web interface: every function, from result lookup to fee payment, is consumed through the API. An unreliable, unauthenticated or unthrottled API is therefore a single point of failure for all 40 colleges and for the app.

11.4 Multi-Tenant Security and Isolation

With 40 colleges sharing one platform, a flaw could expose one college's records to another. Main concerns: tenant data leakage, one tenant's traffic starving others (noisy neighbour), stolen credentials and fee-payment fraud. Required controls:

- A tenant identifier in every token (OAuth 2.0 / JWT claim) checked on each request.
- Row-level security or schema-per-tenant in the database; separate encryption keys per tenant.
- Per-tenant rate limits and quotas at the API gateway.
- Idempotency keys and signed requests on payments; audit logs per tenant.
- A WAF and regular penetration tests.

11.5 Evaluation: Can the Original Design Handle 10× Load?

Assumptions (illustrative): current peak of 2,700 concurrent users at 0.5 requests per second gives 1,350 req/s; one instance sustains 140 req/s at a safe 70% load; the Task 8 design caps auto-scaling at 20 instances.

```
\text{Peak demand} = 1350 \times 10 = 13{,}500 \text{ req/s}
```

```
\text{Original capacity} = 20 \times 140 = 2{,}800 \text{ req/s}
```

The original cap covers about 21% of the new peak even before a CDN. The stateless web tier can scale, but the cap, the single-primary database and the lack of caching and queuing would fail first. **Conclusion: the service models are sound, but the capacity limits are not.**

11.6 Evaluation: Availability of 99.9%

99.9% availability allows 0.001 × 8,760 = 8.76 hours of downtime a year (about 43.8 minutes a month). Components in series multiply their availabilities:

```
A = A_{CDN} \times A_{LB} \times A_{GW} \times A_{app} \times A_{DB}
```

Using typical SLA-style figures as assumptions: CDN/WAF 99.99%, load balancer 99.99%, a single-instance gateway 99.95%, app tier across two zones 99.99% and a primary-plus-standby database 99.95%:

$$A = 0.9999 \times 0.9999 \times 0.9995 \times 0.9999 \times 0.9995 \approx 0.9987$$

This is 99.87%, or about 11.4 hours of downtime a year, which **misses the target**. The weak links are the gateway and the database; the DR region exists but failover is manual (RTO about 1 hour), and a regional failure is not covered automatically.

11.7 Evaluation: Cost and Alternative Services

Always-on capacity for the new peak would be wasteful, so elasticity, a CDN and caching are essential for cost control. Better-suited services than the original design: a **managed API gateway/API management service** (authentication, quotas, analytics), **MBaaS** for the mobile backend, **managed container orchestration** for faster scale-out, a **managed cache and message queue** (to absorb fee-payment bursts) and **serverless functions** for notifications and receipts. SaaS email and library are unaffected.

11.8 Decision: Modify the Design

The decision is **MODIFY** (not retain, not redesign). The hybrid structure, the service-model mix and the container approach remain valid, so a redesign would waste effort. Specific changes are needed to capacity, resilience and tenancy, shown in Figure 11.1 with the changed or added parts highlighted.

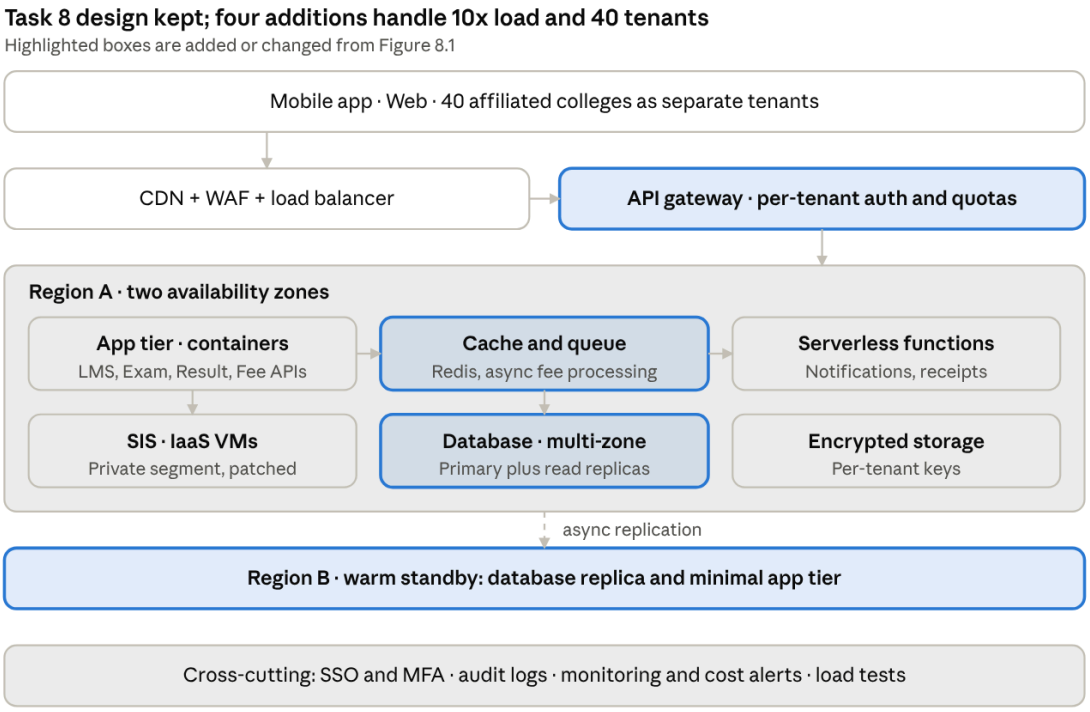


Figure 11.1 · modified architecture, 4 additions

Figure 11.1: Modified architecture for 10× load and 40 tenants. Highlighted boxes are new or changed compared with Figure 8.1.

11.9 What Changes in the Service Mix and Virtualization Approach

Area	Original (Task 8)	Modified
API entry	Gateway as part of the edge	Managed, multi-zone API gateway with per-tenant authentication and quotas
Web tier	PaaS containers, cap of 20 instances	Container platform, scaling cap raised to about 80, scheduled scaling before peaks
Burst handling	None	CDN offload, managed cache, message queue for payments
Database	DBaaS primary plus standby	Multi-zone DBaaS with read replicas; row-level tenant security
DR	DRaaS, manual failover	Warm standby in a second region, automated health-based failover
Virtualization	VMs for SIS, containers for web	Unchanged: SIS stays in VMs; tenant-facing APIs run in containers with resource limits
Notifications	In application	Serverless functions (FaaS)

11.10 Evidence-Based Defence: Capacity and Availability

Capacity. With a CDN absorbing 60% of requests (assumption), origin demand is 13,500 × 0.4 = 5,400 req/s:

```
N = \lceil 5400 / 140 \rceil = 39 \text{ instances per zone}
```

To survive the loss of one of two zones, each zone is sized for the full peak, giving 78 instances at peak. If peaks last about 20 days a year and normal operation needs about 6 instances, the average is (78 × 20 + 6 × 345) / 365 ≈ 10 instances, roughly 87% fewer instance-hours than running 78 all year.

Availability. Making the gateway and database multi-zone with automatic failover (assumed 99.99% each) gives:

```
A = 0.9999^{5} \approx 0.9995
```

That is 99.95%, about 4.4 hours of downtime a year, within the 8.76-hour allowance. The warm standby region also covers a regional outage within an agreed recovery time.

11.11 Cost, Security and Trade-offs

Illustrative annual additions in ₹ lakh: CDN/WAF 2.0, API gateway 2.5, extra peak compute 4.0, cache and queue 1.5, multi-zone database with replicas 2.5, warm standby region 1.5 = **14.0**. One-time redesign, security testing and load testing add ₹6 lakh.

₹ lakh	Original (Option C)	Modified
One-time	17.0	23.0
Annual running	31.4	45.4
3-year total	111.2	159.2

The extra ₹14 lakh a year is about ₹0.35 lakh per college if shared equally among 40 colleges, so a subscription fee can recover it, supporting the limited-budget goal. **Security** improves through tenant-aware tokens, quotas and per-tenant keys. **Trade-offs:** greater complexity, deeper dependence on a single provider's managed services (lock-in), and migration effort, which are reduced by using containers, open API standards and infrastructure-as-code. These figures are assumptions and must be confirmed through load tests and provider quotes.

11.12 Conclusion

The original design has the right structure, but its capacity cap, single-primary database and manual failover cannot meet 10× load and 99.9% availability. Modifying it with a managed multi-zone API gateway, CDN and cache, a queue, read replicas, tenant isolation and a warm standby region meets both targets at a manageable extra cost, and the recommended approach remains a PaaS/SaaS-led hybrid cloud.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50